

# Musion

MANUAL COMPLETO PARA ESTUDO, APRESENTACAO E DEFESA TECNICA

## Musion: arquitetura, produto e funcionamento

Documento guia para explicar o Musion de ponta a ponta: proposta do produto, backend NestJS, banco MySQL, aplicativo mobile React Native/Expo, recomendacoes, sensores, QA, CI/CD e versao web de apoio.

### PROJETO

#### Musion

Rede social mobile para reviews musicais.

### ORDEM DO DOCUMENTO

Produto, backend, mobile, dados/IA, QA, operacao e web por ultimo.

React Native

Expo

NestJS

Prisma

MySQL

Spotify API

Last.fm

Google Sign-In

## COMO USAR ESTE PDF

00

# Mapa de estudo

**Leitura recomendada:** comece pela visao geral, entenda a arquitetura, estude o backend, depois o mobile, e deixe a web para o fim. Se voce precisa apresentar, use as frases prontas no final de cada bloco.

1. Visao geral do produto

2. Requisitos atendidos

3. Arquitetura geral

4. Backend NestJS

5. Banco de dados e Prisma

6. Autenticacao e seguranca

7. Recomendacoes e APIs musicais

8. Aplicativo mobile

9. Fluxos principais do app

10. Sensor, compartilhamento e recursos nativos

11. Moderacao e admin

12. QA, CI/CD e performance

13. Como rodar e configurar

14. Versao web

15. Como explicar em apresentacao

**IDEIA CENTRAL**

O Musion transforma albuns em objetos sociais: cada album pode ter reviews, notas, comentarios, media da comunidade, previews e discussoes.

**DEFESA TECNICA EM UMA FRASE**

O projeto combina app mobile hibrido, backend proprio, banco SQL, APIs musicais, recomendacao inteligente, sensor do dispositivo e pipeline de QA.

## PRODUTO

## 01

# 1. Visao geral do Musion

O Musion e uma rede social para amantes de musica, focada em reviews de albuns. Diferente de um streaming, o objetivo principal nao e apenas tocar musica: e registrar opinioes, dar notas, organizar historico pessoal, descobrir albuns pela comunidade e conversar com outros usuarios.

**PUBLICO-ALVO**

Ouvintes que escutam albuns completos, gostam de avaliar obras e querem descobrir musica por opiniao social.

**PROBLEMA**

Streamings mostram catalogo, mas nao organizam bem reviews, notas, comentarios e comunidade por album.

**SOLUCAO**

Um app mobile social que une catalogo musical, reviews, feed, perfis, chat e recomendacoes.

## Principais funcionalidades

- Cadastro, login com email/senha, login com Google e redefinicao de senha por email.
- Feed social de reviews com curtidas, comentarios, compartilhamento e abertura do post completo.
- Busca de albuns e usuarios.
- Criacao e edicao de reviews com nota de 0 a 10.
- Pagina de album com capa, metadados, faixas, previews, link para Spotify e reviews da comunidade.
- Trending com albuns em alta, lancamentos e recomendacoes personalizadas.
- Perfil com avatar, bio, reviews, seguidores e seguindo.
- Chat entre usuarios e notificacoes de interacoes.
- Bloqueio, denuncia, solicitacao de exclusao e painel admin para moderacao.
- Uso de acelerometro para controlar o slider de nota.

**Como falar:** "O Musion e uma camada social em cima do habito de ouvir albuns. Ele nao compete com o Spotify para tocar musica; ele complementa a experiencia com opiniao, comunidade, historico e descoberta."

## REQUISITOS ACADEMICOS

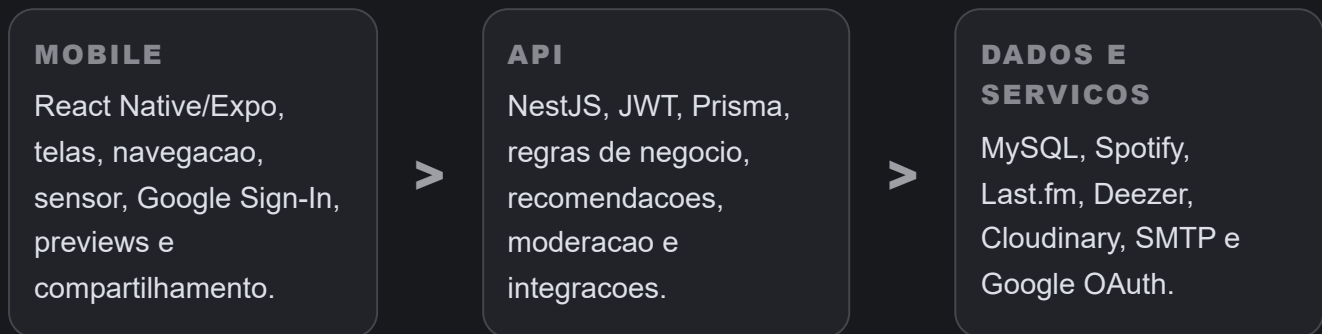
## 02

## 2. O que o projeto atende

| AREA               | COMO O MUSION ATENDE                                                         | EVIDENCIA                                                           |
|--------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Planejamento       | Publico, problema, solucao e prototipagem definidos.                         | Figma, relatorios e fluxo funcional no app.                         |
| Mobile Development | Aplicativo hibrido em React Native com Expo.                                 | <code>musion-mobile/</code>                                         |
| Banco de dados     | Persistencia em MySQL com Prisma ORM.                                        | <code>api/prisma/schema.prisma</code>                               |
| IoT / sensores     | Acelerometro do celular controla a nota da review.                           | Tela AddReview com Expo Sensors.                                    |
| Big Data & IA      | Motor de recomendacao hibrido via API usando dados internos e APIs externas. | Dashboard, Spotify, Last.fm e scoring no backend.                   |
| QA                 | Testes backend, validacao mobile, plano JMeter e builds automatizados.       | <code>.github/workflows</code> e <code>api/tests/performance</code> |
| CI/CD              | Pipeline GitHub Actions e workflow EAS para APK Android.                     | <code>ci.yml</code> e <code>eas-android-preview.yml</code>          |

**Observacao importante:** Game Development nao se aplica, porque o Musion e um projeto mobile, nao um jogo. Os requisitos de Unity, Godot, AR e NPCs ficam fora do escopo escolhido.

## 3. Como as partes se conectam



### Estrutura de pastas

```
C:\Users\Daniel\Desktop\Musion\musion
api/           Backend NestJS + Prisma + MySQL
musion-mobile/ Aplicativo principal React Native/Expo
musion-client/ Versao web em React/Vite
.github/       Workflows de QA e build
```

### Fluxo padrao de uma acao

1. Usuario interage com uma tela do app, por exemplo curtir uma review.
2. O app chama a API com Axios e envia o token JWT no header Authorization.
3. O AuthGuard valida o token e identifica o usuario.
4. O service do modulo executa a regra de negocio.
5. O Prisma grava ou consulta o MySQL.
6. A API retorna JSON e o app atualiza a interface.

**Como explicar:** "A arquitetura separa interface, regra de negocio e dados. O mobile nao acessa banco diretamente; ele conversa com a API, e a API centraliza seguranca, integracoes e persistencia."

# 4. Backend NestJS

O backend fica em `api/` e foi construido com NestJS. Ele e responsavel por autenticacao, regras sociais, integracoes musicais, recomendacoes, email, chat, notificacoes, moderacao e acesso ao banco via Prisma.

### TECNOLOGIAS

- NestJS
- TypeScript
- Prisma
- MySQL
- JWT
- Passport
- bcrypt
- Nodemailer

### PADRAO INTERNO

Controllers recebem requisicoes, services executam regras de negocio, PrismaService acessa o banco, DTOs validam dados de entrada e Guards protegem rotas sensiveis.

## Modulos principais

| MODULO        | RESPONSABILIDADE                                                         |
|---------------|--------------------------------------------------------------------------|
| auth          | Cadastro, login, Google, JWT, reset de senha e validacao do usuario.     |
| users         | Perfil, edicao, avatar, seguidores, busca, senha, exclusao e role admin. |
| reviews       | Criar, editar, listar e remover reviews; reviews por album; comentarios. |
| dashboard     | Feed, tendencias, curtidas e recomendacoes personalizadas.               |
| spotify       | Busca de albuns, detalhes, lancamentos, faixas, links e previews.        |
| chat          | Conversas entre usuarios, busca de usuarios no chat e mensagens.         |
| notifications | Notificacoes de curtidas, comentarios e seguidores.                      |
| moderation    | Denuncias, bloqueios e rotas administrativas de analise.                 |
| legal         | Termos, politica de privacidade e solicitacoes publicas de exclusao.     |

## Rotas de referencia

```
POST /auth/register
POST /auth/login
POST /auth/google
POST /auth/forgot-password
POST /auth/reset-password

GET /users/me
PATCH /users/me
PATCH /users/me/password
DELETE /users/me
GET /users/search
GET /users/profile/:id
POST /users/:id/follow
DELETE /users/:id/follow

POST /reviews
GET /reviews/me
GET /reviews/album/:albumId
PATCH /reviews/:id
DELETE /reviews/:id
GET /reviews/:id/comments
POST /reviews/:id/comments

GET /dashboard
GET /dashboard/feed
POST /dashboard/review/:id/like
GET /dashboard/recommend/recent
GET /dashboard/recommend/second
GET /dashboard/recommend/third
```

## 5. Banco de dados e Prisma

O banco e MySQL, executado localmente por Docker ou instalacao propria. O Prisma define os modelos, gera o cliente TypeScript e aplica migrations. Isso evita SQL manual espalhado pelo projeto e deixa o schema versionado.

### Modelos principais

| MODELO              | O QUE GUARDA                                                                  | RELACOES IMPORTANTES                                               |
|---------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------|
| User                | Conta, email, username, senha criptografada, displayName, avatar, bio e role. | Reviews, follows, likes, comments, reports, blocks, notifications. |
| Review              | Texto, nota, albumId, nome do album, capa, artista e ano.                     | Pertence a User; tem likes, comments e notifications.              |
| Follow              | Relacao follower/following.                                                   | Permite seguidores e seguindo.                                     |
| Like                | Curtida de usuario em review.                                                 | Chave composta userId + reviewId.                                  |
| Comment             | Comentario em review.                                                         | Pertence a User e Review.                                          |
| Notification        | Evento recebido pelo usuario.                                                 | Referencia sender, review e comment quando aplicavel.              |
| Chat                | Mensagens entre usuarios usando sessionId e role.                             | Historico de conversa.                                             |
| UserBlock           | Bloqueio entre usuarios.                                                      | Impede interacoes indesejadas.                                     |
| Report              | Denuncias de usuario, review ou comentario.                                   | Fila de moderacao.                                                 |
| DataDeletionRequest | Pedidos publicos de exclusao de dados.                                        | Atende privacidade e requisitos legais.                            |

**Atualizacao recente:** o campo `role` foi adicionado em User. Ele permite diferenciar usuarios comuns de administradores no painel de moderacao.



## Comandos importantes

```
cd api  
npx prisma generate  
npx prisma migrate deploy  
npx prisma studio
```

## 6. Autenticacao e conta

### Login normal

O usuario informa email e senha. O backend busca o usuario pelo email, compara a senha com bcrypt e, se estiver correto, gera um JWT. O token e enviado ao app e salvo localmente para chamadas futuras.

### Cadastro

O cadastro exige nome, username, email e senha. O backend normaliza email e username, valida duplicidade, criptografa a senha com bcrypt e cria o usuario no MySQL.

### Login com Google

No mobile, o Google Sign-In nativo retorna um idToken. O backend valida esse token usando os Client IDs configurados. Se o email ja existe, ele entra na conta; se nao existe, o backend cria uma conta com nome, email, avatar e username unico.

### Redefinicao de senha

1. Usuario informa o email.
2. API gera um codigo temporario de 6 digitos.
3. Codigo e armazenado com hash e expiracao.
4. Nodemailer envia o email com identidade visual do Musion.
5. Usuario informa codigo e nova senha.
6. Backend valida codigo, atualiza senha e limpa o token.

### Rotas administrativas

O acesso admin usa `AdminGuard`. Ele permite acesso se o usuario tiver `role = ADMIN` ou se o ID estiver no fallback `MODERATION_ADMIN_USER_IDS`. Esse fallback serve para promover o primeiro administrador.

## 7. Recomendacoes e APIs musicais

A tela Trending e as recomendacoes dependem de um conjunto de dados internos e externos. O objetivo e evitar recomendacoes repetitivas e sugerir albuns que tenham relacao com o gosto do usuario e com a atividade da comunidade.

### SINAIS INTERNOS

- Reviews criadas pelo usuario.
- Notas altas e historico recente.
- Albuns ja avaliados, para evitar repeticao.
- Likes, comentarios e reviews da comunidade.

### SINAIS EXTERNOS

- Spotify para busca, metadados, lancamentos e links.
- Last.fm para tags, artistas similares e apoio semantico.
- Fallback de preview quando Spotify nao entrega trecho de faixa.

### Como defender como IA / Big Data

O projeto usa um motor de recomendacao hibrido consumido por API. Ele analisa dados coletados no app e dados de bases musicais externas para calcular sugestoes. Mesmo sem treinar um modelo proprio pesado, existe uma camada inteligente de scoring, ranking, filtros e diversificacao.

**Limite real de API:** Spotify pode retornar 429 quando muitas requisicoes sao feitas. Por isso o backend precisa de cache, reducao de chamadas repetidas e tratamento de fallback. Isso e uma boa oportunidade para explicar preocupacao com performance e robustez.

# 8. Mobile React Native/Expo

O mobile e o produto principal do Musion. Ele fica em `musion-mobile/`, usa Expo SDK 54, React Native 0.81, React Navigation e componentes nativos para camera/galeria, sensores, audio, compartilhamento e teclado.

### DEPENDENCIAS IMPORTANTES

- expo
- react-navigation
- axios
- async-storage
- expo-av
- expo-sensors
- expo-sharing
- view-shot
- google-signin

### NAVEGACAO

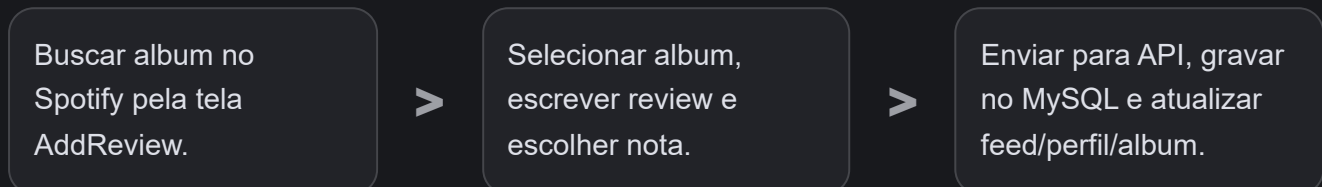
O app usa Stack Navigator para telas completas e Bottom Tab Navigator para Feed, Search, Trending e Chat. A splash e exibida antes da navegacao principal.

## Telas principais

| TELA               | FUNCAO                                                                                    |
|--------------------|-------------------------------------------------------------------------------------------|
| LoginScreen        | Login, cadastro, Google, reset de senha e mostrar/ocultar senha.                          |
| FeedScreen         | Timeline social com reviews, likes, comentarios, compartilhamento e acesso a perfil/post. |
| SearchScreen       | Busca por albuns e usuarios.                                                              |
| TrendingScreen     | Tendencias, lancamentos e recomendacoes personalizadas.                                   |
| AddReviewScreen    | Busca album, escreve review, nota com slider e controle por acelerometro.                 |
| AlbumDetailsScreen | Detalhes do album, faixas, previews, link Spotify e reviews da comunidade.                |
| PostScreen         | Review completa e comentarios.                                                            |
| ProfilePage        | Perfil, avatar, bio, seguidores, seguindo e reviews do usuario.                           |
| ChatScreen         | Conversas e envio de mensagens entre usuarios.                                            |
| SettingsScreen     | Configuracoes, senha, exclusao, termos, privacidade e bloqueados.                         |

## 9. Como as funcionalidades funcionam

### Fluxo: criar uma review



### Fluxo: abrir pagina de album

1. Usuario clica em capa, titulo ou resultado de busca.
2. App carrega detalhes do album na API.
3. Backend consulta Spotify e complementa dados com reviews do Musion.
4. Tela mostra capa, artista, ano, faixas, media, previews e reviews.

### Fluxo: social

- Curtir uma review cria ou remove um registro em Like.
- Comentar cria Comment e pode gerar Notification.
- Seguir cria Follow e notifica o usuario seguido.
- Chat cria mensagens com sessionId compartilhado entre os dois usuarios.

### Fluxo: notificacoes

O backend registra eventos relevantes como LIKE, FOLLOW e COMMENT. A tela de notificacoes lista esses eventos e permite marcar como lido. Isso aproxima o app de uma rede social real.

# 10. Sensor, audio e compartilhamento

## Acelerometro no slider de nota

O recurso de sensor fica na tela de adicionar review. Quando ativado, o acelerometro do celular captura a inclinacao do aparelho e transforma esse valor em nota no slider. Isso demonstra coleta de dado fisico em tempo real e aplicacao direta na interface.

**Frase de defesa:** "Na parte de sensores, usamos o hardware do proprio celular. O acelerometro controla a nota da review em tempo real, entao o dado fisico influencia diretamente o comportamento do app."

## Previews de faixa

A pagina de album tenta tocar previews das faixas com `expo-av`. Quando o Spotify nao fornece preview, o backend tenta uma alternativa externa. Se nao houver trecho disponivel, o app ainda oferece o link do album no Spotify.

## Compartilhamento de review

O app monta um card visual de review com capa, titulo, artista, nota, texto e assinatura do usuario. Esse card e capturado como imagem com view-shot e compartilhado via Expo Sharing. Assim o conteudo do Musion pode circular fora do app sem perder identidade visual.

## Teclado e experiencia mobile

Telas como Post e Chat foram ajustadas para trabalhar com teclado em Android, evitando que inputs fiquem escondidos pela barra de navegacao ou pelo teclado virtual.

# 11. Moderacao, bloqueio e admin

Como o Musion tem interacao social, o projeto inclui mecanismos de seguranca: denuncia, bloqueio, solicitacao de exclusao e painel administrativo.

## DENUNCIA

Usuarios podem denunciar perfis, reviews ou comentarios. A denuncia fica salva em Report com targetType, targetId, reporterId, reason, details e status.

## BLOQUEIO

Bloqueios sao salvos em UserBlock. O sistema remove conexoes de follow e evita interacoes indesejadas entre usuarios.

## Painel admin

A versao web possui a rota `/admin/moderation`. Ela permite analisar denuncias, atualizar status, ver solicitacoes de exclusao e gerenciar usuarios administradores.

## Como criar um admin

1. Defina temporariamente `MODERATION_ADMIN_USER_IDS` no backend com o ID do seu usuario.
2. Acesse a web, entre na conta e abra `/admin/moderation`.
3. No painel, busque seu usuario e use "Tornar admin".
4. Depois disso, o banco tera `role = ADMIN` e o fallback pode ficar vazio.

```
MODERATION_ADMIN_USER_IDS="1"
```

## 12. QA, CI/CD e performance

### Validador atual

| CAMADA      | VALIDACAO                         | COMANDO OU ARQUIVO                                |
|-------------|-----------------------------------|---------------------------------------------------|
| Backend     | Testes unitarios com Jest.        | <code>npm test -- --runInBand</code>              |
| Backend     | Build NestJS.                     | <code>npm run build</code>                        |
| Mobile      | Validacao JSX.                    | <code>npm run validate</code>                     |
| Performance | Plano JMeter.                     | <code>api/tests/performance/musion-api.jmx</code> |
| CI/CD       | GitHub Actions para API e mobile. | <code>.github/workflows/ci.yml</code>             |
| APK         | Build Android preview com EAS.    | <code>eas-android-preview.yml</code>              |

### Pipeline principal

O workflow `Musion QA` instala dependencias, gera Prisma Client, roda testes, faz build da API, instala dependencias mobile e executa a validacao JSX.

### Build Android

O workflow EAS Android Preview executa validacao do app e dispara build Android preview usando o token da Expo configurado nos secrets do GitHub.

**Como explicar:** "O QA nao esta limitado a teste manual. O projeto tem teste, build, validacao mobile, performance planejada com JMeter e automacao de build Android."



# 13. Como rodar e configurar

## Banco com Docker

```
cd C:\Users\Daniel\Desktop\Musion\musion
docker compose up -d
```

## Backend

```
cd C:\Users\Daniel\Desktop\Musion\musion\api
npm install
npx prisma generate
npx prisma migrate deploy
npm run start:dev
```

## Mobile

```
cd C:\Users\Daniel\Desktop\Musion\musion\musion-mobile
npm install
npx expo start
```

## APK preview

```
cd C:\Users\Daniel\Desktop\Musion\musion\musion-mobile
npx eas-cli@latest build -p android --profile preview --clear-cache
```

## Variaveis de ambiente mais importantes

| VARIAVEL                   | USO                                               |
|----------------------------|---------------------------------------------------|
| DATABASE_URL               | Conexao MySQL usada pelo Prisma.                  |
| JWT_SECRET                 | Assinatura dos tokens JWT.                        |
| SPOTIFY_CLIENT_ID / SECRET | Busca de albuns, faixas, lancamentos e metadados. |
| LASTFM_API_KEY             | Tags e apoio para recomendacoes.                  |
| CLOUDINARY_*               | Upload de avatar.                                 |
| SMTP_*                     | Envio do codigo de redefinicao de senha.          |
| GOOGLE_WEB_CLIENT_ID       | Validacao do login com Google no backend.         |
| MODERATION_ADMIN_USER_IDS  | Fallback para acesso admin inicial.               |
| EXPO_PUBLIC_API_URL        | URL da API usada pelo app mobile.                 |
| EXPO_PUBLIC_GOOGLE_*       | Client IDs usados pelo Google Sign-In no mobile.  |

# 14. Versao web

A web fica em `musion-client/` e usa React com Vite. Ela e menos importante que o mobile na defesa principal, mas hoje funciona como extensao do produto: dashboard publico, autenticacao, navegacao por albuns/perfis, reviews, chat, configuracoes e painel administrativo.

## O que existe na web

- Landing/dashboard publico para visitante conhecer albuns aclamados e buscar conteudo.
- Login/cadastro e login com Google quando configurado.
- Layout autenticado com busca central, perfil, menu e nova review.
- Feed, perfil, album page, post page, chat, notificacoes e configuracoes.
- Painel admin em `/admin/moderation`.

## Por que ela fica por ultimo

O requisito principal do projeto e mobile. A web e uma vitrine e uma ferramenta operacional, especialmente para admin/moderacao. Na apresentacao, use a web para mostrar painel admin ou dashboard publico se houver tempo, mas concentre a demonstracao no app mobile.

## Como rodar

```
cd C:\Users\Daniel\Desktop\Musion\musion\musion-client
npm install
npm run dev
```

# 15. Como explicar o Musion

## Resumo em 30 segundos

"O Musion e uma rede social mobile para reviews de albuns. O usuario cria uma conta, avalia discos, comenta, curte, segue perfis, conversa por chat e recebe recomendacoes personalizadas. Por tras, existe um backend NestJS com MySQL, Prisma, JWT, integracoes com Spotify e Last.fm, reset de senha por email, moderacao e CI/CD."

## Ordem ideal no video

1. Splash, login e cadastro.
2. Feed e abertura de uma review.
3. Criar review buscando album.
4. Mostrar acelerometro controlando nota.
5. Pagina de album com faixas, preview e Spotify.
6. Trending e recomendacoes.
7. Perfil, seguidores, chat e notificacoes.
8. Compartilhamento de review como imagem.
9. Moderacao, bloqueio e configuracoes.
10. Arquitetura backend, banco, APIs e QA.
11. Web por ultimo: dashboard publico e admin.

## Perguntas que podem surgir

| PERGUNTA                          | RESPOSTA BOA                                                                                         |
|-----------------------------------|------------------------------------------------------------------------------------------------------|
| Por que isso não é só um Spotify? | Porque o foco é opinião social, histórico de reviews, notas, comunidade e descoberta por pessoas.    |
| Onde entra sensor?                | No acelerômetro que controla o slider da nota em tempo real.                                         |
| Onde entra IA/dados?              | No motor de recomendação híbrido via API, que usa histórico do usuário, comunidade e dados externos. |
| Onde os dados ficam?              | No MySQL, acessado por Prisma no backend NestJS.                                                     |
| Como o app é seguro?              | Senha com bcrypt, JWT, guards, reset por código, roles admin e rotas protegidas.                     |
| Como funciona moderação?          | Denúncias entram em Report, bloqueios em UserBlock e admins analisam no painel web.                  |

Documento gerado para apoiar estudo e defesa do Musion. Use como guia técnico e roteiro de fala.